

Practical No: 03

Aim: Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Software Requirement: Python IDE, Google Colab, VS Code

Theory:

Introduction:

In many real-world applications, datasets have a large number of features(dimensions) which can make data analysis and visualization complex and computationally expensive.

Dimensionality Reduction techniques are used to reduce the number of input variables while retaining the most important information.

Principal Component Analysis (PCA) is one of the most widely used linear dimensionality reduction techniques in machine learning and data science.

Definition of PCA:

Principal Component Analysis (PCA) is a statistical method that transforms the original correlated features into a new set of uncorrelated features called principal components. These components capture the maximum variance in the data.

Objectives of PCA:

- Reduce the dimensionality of data
- Retain maximum variance/information from original data
- Remove redundancy and noise
- Improve efficiency for machine learning models and visualization

Key Concepts:

Covariance Matrix:

Measures the relationship (correlation) between different features in the data.

Eigen values and Eigen vectors:

Eigen vectors define the direction of new feature axes (principal components), and eigen values indicate the magnitude (importance) of these directions.

Principal Components:

The new axes after transformation.

The first principal component accounts for the most variance, followed by the second, and soon.

Variance Explained:

A measure of how much information (variance) each principal component retains from the original data.

Steps in PCA:

- Standardize the dataset.
- Scale the data so that each feature has zero mean and unit variance.
- Compute the covariance matrix.
- Calculate eigen values and eigen vectors.
- Sort eigenvectors by decreasing eigenvalues(importance).
- Select top-k eigen vectors to form a new matrix.
- Transform the original data using this matrix (i.e., project it onto the new axes).

Advantages of PCA:

- Reduces computational cost
- Eliminates multicollinearity
- Useful for visualization (e.g., 2D or 3D plots)
- Enhances model performance by removing noise

Limitations:

PCA is a linear technique and may not capture nonlinear relationships.

Components are not interpretable in terms of original features.

Sensitive to the scaling of data.

Applications:

- Image compression
- Face recognition

- Genomics and bioinformatics
- Exploratory data analysis
- Preprocessing for machine learning models

Code:

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer
from mpl_toolkits.mplot3d import Axes3D

# Load and prepare data
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
X_scaled = StandardScaler().fit_transform(X)

# Apply PCA
pca = PCA(n_components=3)
X_pca = pca.fit_transform(X_scaled)

# Print info
print("Classes:", data.target_names)
print("Features:", data.feature_names)
print("PCA Shape:", X_pca.shape)
print("Principal Components:\n", pca.components_)

# 2D Plot
plt.figure(figsize=(6,6))
plt.scatter(X_pca[:,0], X_pca[:,1], c=data.target, cmap='plasma')
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.title('2D PCA'); plt.tight_layout(); plt.show()

# 3D Plot
```

```
fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_pca[:,0], X_pca[:,1], X_pca[:,2], c=data.target, cmap='plasma')
ax.set_xlabel('PC1'); ax.set_ylabel('PC2'); ax.set_zlabel('PC3'); ax.set_title('3D PCA')
plt.tight_layout(); plt.show()
```

Conclusion:

Hence, I have successfully implemented dimensionality reduction using Principle Component Analysis (PCA) method.

Viva Questions

What is the purpose of PCA?

What is the difference between PCA and feature selection?

What are eigen values and eigen vectors in PCA?

Can PCA be used for classification?

What are the limitations of PCA?